containerId, cargoId, cargoKind, and cargoSize, separated by spaces, and press Enter.

NOTE You haven't learned how to interact with a cluster from the outside. Sending messages through the console is not how you would usually do this. This solution is implemented for you so you can play with it. Chapter 11 covers how to send messages to a cluster through HTTP and gRPC.

When you send a message to the entity, an event is generated and stored in the database. Here's how three messages are sent to the SPContainer:

```
$ 9 456 sack 22       ❶
$ 9 459 bigbag 15     ❶
$ 11 499 barrel 120   ❶
```

❶ Enter these values, pressing Enter after each one.

These messages add cargos 456 and 459 to container 9 and cargo 499 to container 11; table 9.1 shows the resulting journal entries in the database table journals_events. (Not all table fields are shown.)

Table 9.1 Journal entries in the database table public.journal_events

ordering	persistence_id	sequence_number	deleted	event_ser_id	event_payload	event_ser_id
1	SPContainer-type-key|9	1	false	33	binary data	33
2	SPContainer-type-key|9	2	false	33	binary data	33
3	SPContainer-type-key|11	3	false	33	binary data	33

The persistence_id field consists of the typeKey and containerId. The sequence_number is an ascending number for each entity. The deleted field allows you to mark events as deleted because in certain cases, for compliance or security reasons, you need to delete the data. (In the event-sourcing approach, the data is not deleted but only marked as such.) The event_payload, the actual CargoAdded message, is stored in cbor-jackson, so it is binary. If you choose the json-jackson binding, you can read the message directly in the table.

If the event_ser_id is between 0 and 40, it means the SerDe used is from an Akka module. For example, 33 is jackson-cbor from the module akka-serialization-jackson.